

Laporan Proyek Mata Kuliah Rekayasa Perangkat Lunak
MATCS: Multi-Agent Traffic Control System

Samuel Tanaka Sibarani (2206059710)
Alfonsus Tanara G (2306267126)
Ade Zaskia Azzahra (2406344353)
Prof. Dr. Ir. Riri Fitri Sari, M.M., M.Sc. (RFS)
Departemen Teknik Elektro, Fakultas Teknik, Universitas Indonesia
2026

Abstract

Laporan ini mendokumentasikan MATCS (Multi-Agent Traffic Control System), sebuah proyek rekayasa perangkat lunak untuk mengembangkan sistem pengendalian lalu lintas berbasis simulasi. Sistem dibatasi pada satu persimpangan dengan empat pendekat, kedatangan kendaraan yang dapat direproduksi, pengendali fixed-time, pengendali queue-aware heuristic, validator keselamatan deterministik, aktuator simulasi, serta metrik evaluasi. Implementasi awal menggunakan Python standard library tanpa koneksi ke perangkat lampu lalu lintas, jaringan publik, atau sistem otoritas. Laporan ini memperbarui rancangan awal dengan bukti implementasi, 13 pengujian otomatis, dan 80 eksperimen terkontrol. Hasil awal menunjukkan bahwa heuristik queue-aware menjanjikan pada permintaan yang tidak seimbang, tetapi belum dapat diklaim lebih baik secara universal. Bab tambahan membahas project plan, UML, rancangan MoU, estimasi COCOMO, dan Function Point Analysis.

Abstract (English)

This report documents MATCS (Multi-Agent Traffic Control System), a software-engineering project for simulation-based traffic control. The current scope is one four-approach intersection with reproducible vehicle arrivals, fixed-time and queue-aware heuristic controllers, a deterministic safety validator, a simulated actuator, and evaluation metrics. The implementation uses only the Python standard library and does not connect to traffic-light hardware, public networks, or authority systems. This report updates the initial design with implementation evidence, 13 automated tests, and 80 controlled experiment runs. Initial results suggest that the queue-aware heuristic is promising under directional imbalance but cannot yet be claimed universally superior. Additional chapters cover the project plan, UML, a draft MoU, COCOMO estimation, and Function Point Analysis.

Contents

1 BAB 1. PENDAHULUAN **4**
1.1 Latar Belakang . . . 4
1.2 Identifikasi Masalah . . . 4
1.3 Rumusan Masalah . . . 4
1.4 Tujuan . . . 4
1.5 Ruang Lingkup dan Batasan . . . 4
1.6 Metodologi . . . 4
1.7 Sistematika Penulisan . . . 4

2 BAB 2. TINJAUAN PUSTAKA **5**
2.1 Adaptive Traffic Control . . . 5
2.2 Multi-Agent Software Architecture . . . 5
2.3 Deterministic Safety Boundary . . . 5
2.4 Estimasi Perangkat Lunak . . . 5

3 BAB 3. PROJECT PLAN DAN UML **5**
3.1 Pendekatan Manajemen Proyek . . . 5
3.2 Struktur Tim dan Tanggung Jawab . . . 5
3.3 Fase dan Milestone . . . 5
3.4 Jadwal dan Definition of Done . . . 6
3.5 UML 1: Use-Case Diagram . . . 6
3.6 UML 2: Class Diagram . . . 7
3.7 UML 3: Sequence Diagram . . . 8
3.8 UML 4: Activity Diagram . . . 9
3.9 UML 5: State-Machine Diagram . . . 10
3.10 UML Tambahan: Component dan Deployment . . . 11

4 BAB 4. ANALISIS, PERENCANAAN, IMPLEMENTASI, DAN EVALUASI SEMENTARA **11**
4.1 4.1 Draft Memorandum of Understanding (MoU) . . . 11
4.1.1 Para Pihak . . . 11
4.1.2 Tujuan . . . 11
4.1.3 Ruang Lingkup dan Kewajiban . . . 11
4.1.4 Keselamatan, Data, dan Kekayaan Intelektual . . . 11
4.1.5 Durasi, Perubahan, dan Terminasi . . . 11
4.2 4.2 COCOMO: Estimasi Awal . . . 12
4.3 4.3 Function Point Analysis . . . 12
4.4 4.4 Analisis Kebutuhan dan Arsitektur . . . 12
4.5 Kebutuhan Fungsional . . . 12

4.6 Kebutuhan Nonfungsional . . . 12
4.7 Arsitektur . . . 13
4.8 4.5 Implementasi Sistem . . . 13
4.9 Lingkungan Pengembangan . . . 13
4.10 Domain Models . . . 13
4.11 Controllers and Safety . . . 13
4.12 Metrics and Batch Runner . . . 13
4.13 4.6 Pengujian dan Validasi . . . 13
4.14 Automated Tests . . . 13
4.15 Acceptance Result . . . 13
4.16 4.7 Hasil Eksperimen dan Pembahasan . . . 13
4.17 Eksperimen . . . 13
4.18 Interpretasi . . . 14
4.19 Kesiapan Fase Berikutnya . . . 14

5 BAB 5. KESIMPULAN DAN SARAN (TEMPORARY) 14
5.1 Kesimpulan Sementara (Temporary) . . . 14
5.2 Saran Pengembangan Sementara (Temporary) . . . 14
5.3 Keterbatasan Saat Ini (Temporary) . . . 14

List of Figures

1 Use-case diagram MATCS. . . . 7
2 Class diagram MATCS. . . . 8
3 Sequence diagram satu simulation tick MATCS. . . . 9
4 Activity diagram simulasi MATCS. . . . 10
5 State-machine diagram fase sinyal MATCS. . . . 11

List of Tables

Daftar Singkatan dan Istilah

- MATCS: Multi-Agent Traffic Control System.
- UML: Unified Modeling Language.
- MoU: Memorandum of Understanding.
- COCOMO: Constructive Cost Model.
- FPA: Function Point Analysis.
- CLI: Command-Line Interface.
- Queue-aware: pengendali yang menggunakan panjang antrean dan waktu tunggu untuk memilih fase berikutnya.
- Shadow mode: mode rekomendasi tanpa mengambil alih sistem lapangan.

1 BAB 1. PENDAHULUAN

1.1 Latar Belakang

Kemacetan perkotaan berubah menurut waktu, pola perjalanan, aktivitas publik, cuaca, dan insiden. Pengaturan lampu lalu lintas yang sepenuhnya tetap dapat memberikan waktu hijau yang tidak sebanding dengan permintaan aktual. Akibatnya, suatu pendekat dapat membentuk antrean sementara pendekat lain belum menggunakan kapasitasnya.

Masalah ini menjadi lebih kompleks ketika beberapa persimpangan berada dalam satu koridor. Keputusan yang mengurangi antrean pada satu persimpangan dapat memindahkan beban ke persimpangan berikutnya. Karena itu, MATCS dirancang sebagai studi rekayasa perangkat lunak yang memisahkan observasi, pembentukan state, proposal pengendali, validasi keselamatan, aktuasi, dan evaluasi.

1.2 Identifikasi Masalah

Masalah utama proyek adalah bagaimana membangun prototipe pengendalian lalu lintas yang dapat diuji secara aman, murah, dan dapat direproduksi. Sistem harus mampu membandingkan kebijakan pengendali pada kondisi permintaan yang sama, mencatat metrik secara konsisten, dan mencegah proposal fase yang melanggar urutan keselamatan.

1.3 Rumusan Masalah

1. Bagaimana membangun simulator satu persimpangan yang deterministik dan mudah ditinjau oleh tiga mahasiswa?
2. Bagaimana membandingkan fixed-time controller dan queue-aware heuristic secara adil?
3. Bagaimana menerapkan batas keselamatan deterministik sebelum aktuasi simulasi?
4. Bagaimana mendokumentasikan project plan, UML, MoU, COCOMO, dan FPA secara konsisten dengan implementasi?

1.4 Tujuan

Tujuan utama adalah mengembangkan dan mengevaluasi vertical slice MATCS untuk satu persimpangan. Tujuan teknisnya adalah menyediakan simulator lokal, controller baseline, heuristic controller, safety validator, metrics collector, automated tests, dan batch experiment runner.

1.5 Ruang Lingkup dan Batasan

Ruang lingkup saat ini meliputi satu persimpangan empat pendekat, empat demand profile, dua controller, synthetic arrivals, simulasi fase, metrik antrean dan delay, serta output CSV/JSON. Sistem tidak mengendalikan lampu lalu lintas nyata, tidak membaca kabinet sinyal, tidak melakukan koneksi authority gateway, dan tidak mengklaim keselamatan untuk deployment.

1.6 Metodologi

Pengembangan dilakukan secara iteratif: definisi kontrak, implementasi vertical slice, pengujian, eksperimen matched-seed, interpretasi hasil, dan revisi dokumentasi. Semua percobaan dapat diulang menggunakan seed, durasi, demand profile, dan controller yang tercatat.

1.7 Sistematika Penulisan

Bab 1 menjelaskan konteks. Bab 2 membahas dasar teori dan teknologi. Bab 3 berisi project plan dan UML. Untuk sementara, materi analisis kebutuhan, arsitektur, MoU, estimasi, implementasi, pengujian, dan hasil ditempatkan sebagai sub-bab pada Bab 4 sampai ada instruksi struktur lebih lanjut dari dosen. Bab 5 berisi kesimpulan sementara.

2 BAB 2. TINJAUAN PUSTAKA

2.1 Adaptive Traffic Control

Adaptive traffic control menggunakan observasi permintaan untuk menyesuaikan keputusan fase. Dalam proyek ini, adaptivitas pertama diwujudkan melalui heuristic queue-aware, bukan model machine learning.

2.2 Multi-Agent Software Architecture

Istilah multi-agent digunakan untuk memisahkan tanggung jawab: observation/state adapter, intersection controller, safety validator, simulated actuator, dan evaluation module. Pada satu persimpangan seluruh komponen berjalan sebagai modul dalam satu proses lokal. Deployment terpisah dan koordinasi antar-persimpangan ditunda.

2.3 Deterministic Safety Boundary

Controller hanya menghasilkan proposal abstrak berupa fase saat ini atau fase berikutnya. SafetyValidator menolak timestamp yang stale dan transisi yang melewati fase. Aturan ini tetap berlaku jika controller pada masa depan diganti optimizer atau learned policy.

2.4 Estimasi Perangkat Lunak

COCOMO dan FPA digunakan sebagai alat perencanaan, bukan sebagai bukti biaya aktual. Nilainya bergantung pada ukuran kode, kompleksitas, tingkat pengalaman tim, dan batasan akademik. Oleh sebab itu, asumsi perhitungan disajikan secara eksplisit pada Bab 4.

3 BAB 3. PROJECT PLAN DAN UML

3.1 Pendekatan Manajemen Proyek

Tim menggunakan proses iteratif ringan dengan Trello sebagai sumber kebenaran project management dan GitHub sebagai sumber kebenaran kode serta dokumentasi. Setiap pekerjaan memiliki tujuan, owner, acceptance evidence, review, dan catatan risiko.

3.2 Struktur Tim dan Tanggung Jawab

Anggota	Tanggung jawab utama
Samuel Tanaka Sibarani	Koordinasi, integrasi report, simulator, eksperimen, dan administrasi project board.
Alfonsus Tanara G	Riset controller, safety boundary, desain eksperimen, dan review teknis.
Ade Zaskia Azzahra	Dokumentasi, UML, project evidence, dan review kebutuhan/presentasi.

3.3 Fase dan Milestone

Fase	Deliverable dan acceptance evidence
Fase 0: Scope	Batas simulation-only, non-goals, risk register, dan repository setup.
Fase 1: Vertical slice	Models, fixed-time controller, validator, simulator, metrics, CLI.
Fase 2: Evidence	Queue-aware controller, demand profiles, 13 tests, CSV/JSON batch results.
Fase 3: Review	UML, report, interpretation, code review, dan demonstrasi lokal.
Fase 4: Future research	Statistical evaluation, calibrated digital twin, lalu possible shadow mode dengan persetujuan terpisah.

3.4 Jadwal dan Definition of Done

Pekerjaan dinyatakan selesai apabila implementasi berjalan, test atau evidence tersedia, anggota lain dapat meninjau hasilnya, dokumentasi diperbarui, dan risiko tindak lanjut dicatat. Fase berikutnya tidak boleh menghapus fixed-time baseline atau safety validator.

3.5 UML 1: Use-Case Diagram

Aktor utama adalah student developer dan lecturer/reviewer. Authority/site stakeholder hanya berinteraksi pada batas future observation approval, bukan sebagai asumsi hak aktuasi.

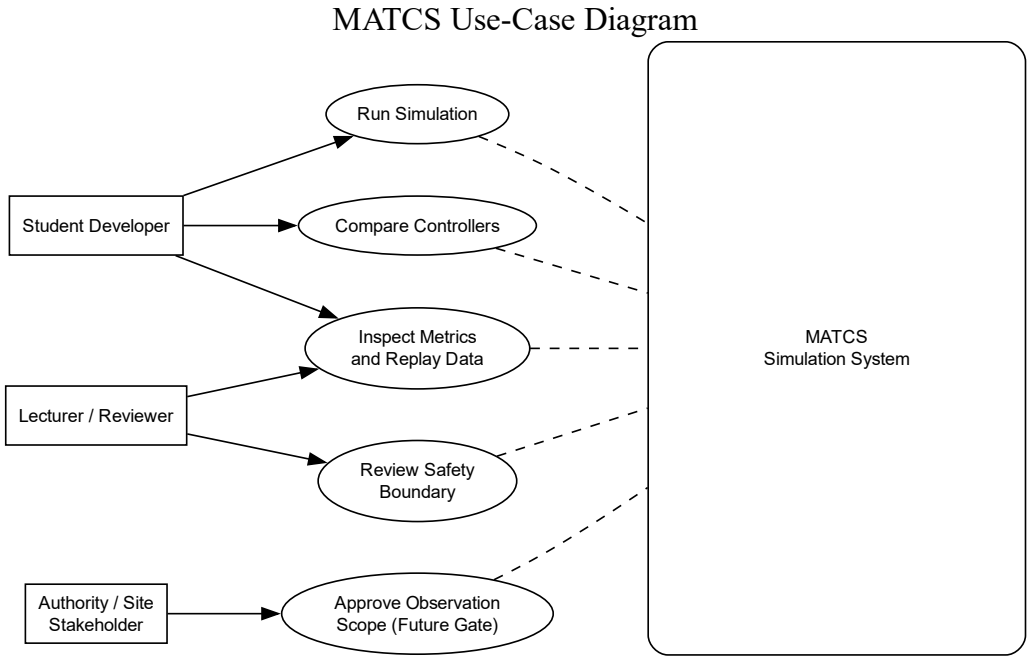


Figure 1: Use-case diagram MATCS.

3.6 UML 2: Class Diagram

Kelas utama adalah ScenarioConfig, DemandProfile, TimingPlan, SimulationState, SignalState, FixedTimeController, QueueAwareController, SafetyValidator, SimulatedActuator, TrafficSimulator, MetricsCollector, dan RunMetrics.

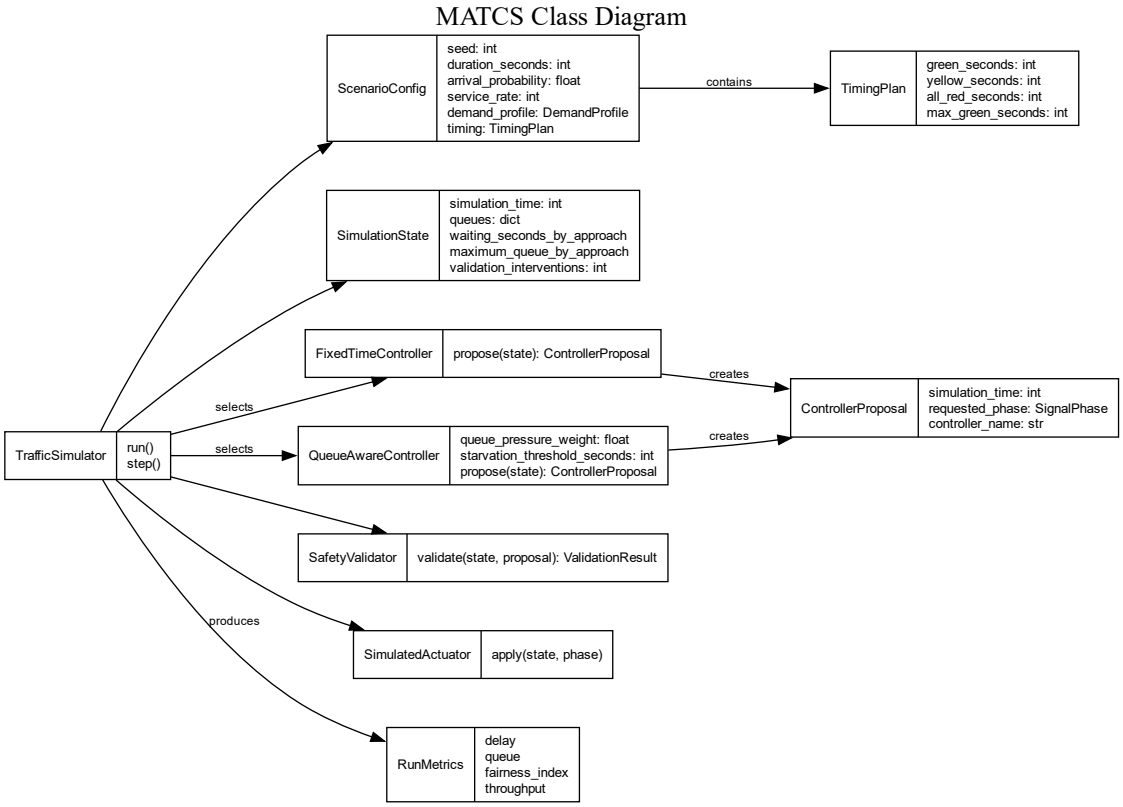


Figure 2: Class diagram MATCS.

3.7 UML 3: Sequence Diagram

Urutan satu tick adalah generate arrivals; controller propose; validator validate; actuator apply jika accepted; move vehicles; update waiting/delay; metrics observe; dan advance simulation time.

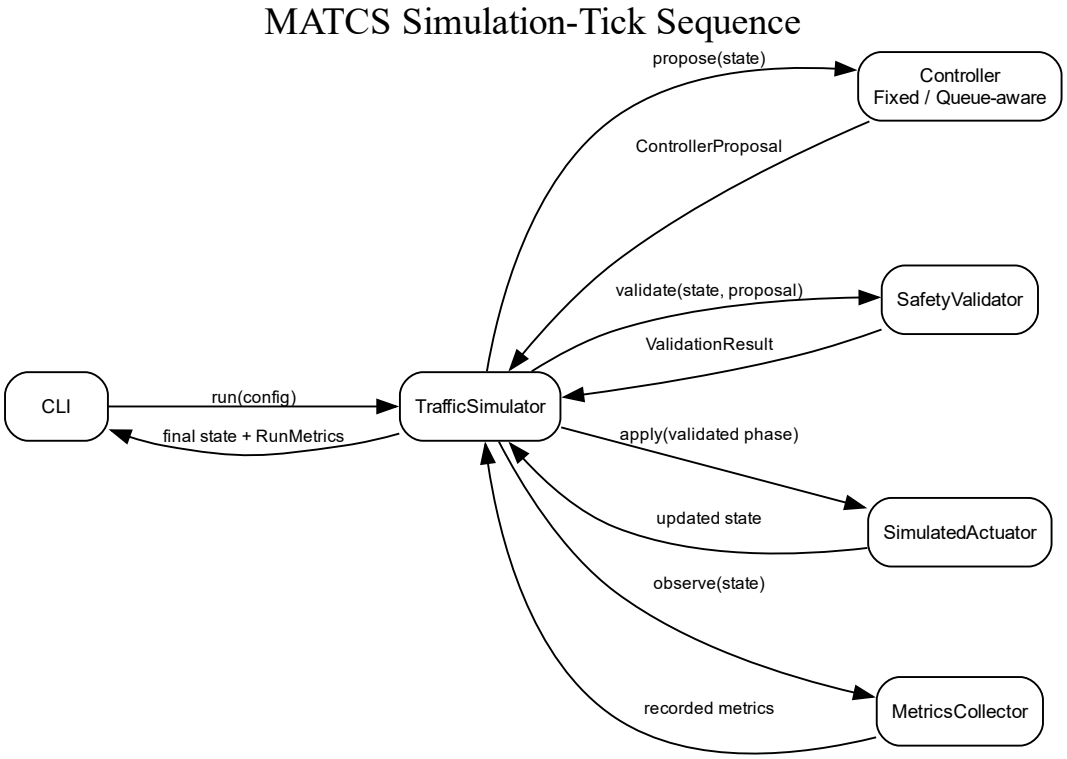


Figure 3: Sequence diagram satu simulation tick MATCS.

3.8 UML 4: Activity Diagram

Aktivitas dimulai dengan konfigurasi seed, durasi, demand profile, dan controller. Sistem melakukan loop tick sampai durasi selesai. Jalur accepted menuju simulated actuator, sedangkan jalur rejected mempertahankan fase aman dan mencatat intervention.

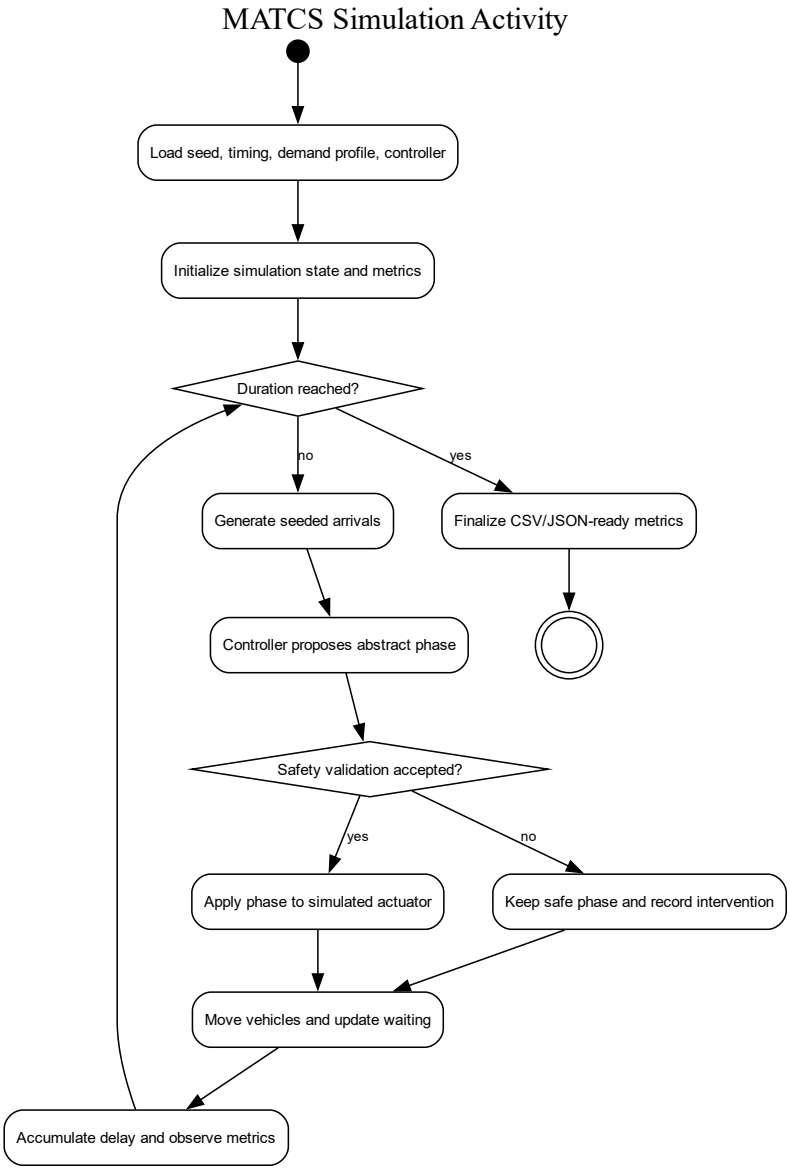


Figure 4: Activity diagram simulasi MATCS.

3.9 UML 5: State-Machine Diagram

Urutan fase adalah North/South Green, North/South Yellow, All-Red to East/West, East/West Green, East/West Yellow, dan All-Red to North/South. Controller tidak dapat melompati yellow atau all-red.

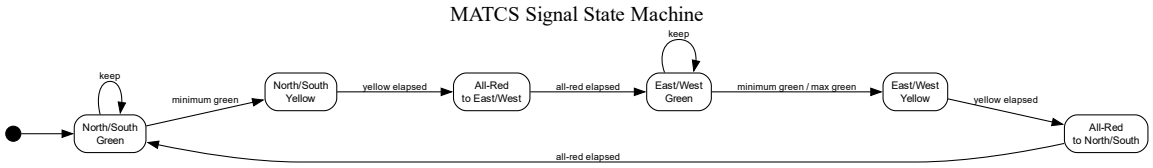


Figure 5: State-machine diagram fase sinyal MATCS.

3.10 UML Tambahan: Component dan Deployment

Komponen tambahan adalah demand generator, observation adapter, state, controller strategy, safety validator, simulated actuator, metrics collector, CLI, dan future authority-approved observation adapter. Deployment saat ini hanya satu laptop dengan satu proses Python dan test runner lokal. Diagram editable lengkap tersedia pada ‘docs/11-uml-models.md‘ di repository.

4 BAB 4. ANALISIS, PERENCANAAN, IMPLEMENTASI, DAN EVALUASI SEMENTARA

4.1 4.1 Draft Memorandum of Understanding (MoU)

Bagian ini adalah template akademik untuk kesepakatan internal kelompok; bukan izin akses infrastruktur publik.

4.1.1 Para Pihak

Pihak A adalah tim mahasiswa MATCS. Pihak B adalah dosen/pembimbing atau pemilik konteks akademik yang ditunjuk. Nama, jabatan, dan tanggal efektif harus diisi setelah disepakati.

4.1.2 Tujuan

MoU mengatur kolaborasi pengembangan software simulation-first, pembagian tugas, review evidence, penggunaan repository, dan batas keselamatan. Tujuan MoU bukan memberikan kewenangan untuk mengendalikan sinyal publik.

4.1.3 Ruang Lingkup dan Kewajiban

Pihak A mengembangkan simulator, controller, tests, eksperimen, UML, dan dokumentasi. Pihak B memberikan review akademik, acceptance criteria, dan umpan balik. Semua pihak menjaga reproducibility, mencatat keputusan, dan tidak mempublikasikan data sensitif tanpa persetujuan.

4.1.4 Keselamatan, Data, dan Kekayaan Intelektual

Aktuator hanya simulasi. Data lapangan, bila suatu saat disetujui, hanya digunakan sesuai izin tertulis, minimisasi data, dan prosedur yang disepakati. Kode dan report dikelola pada repository kelompok; kontribusi anggota dicatat melalui commit, review, dan dokumen.

4.1.5 Durasi, Perubahan, dan Terminasi

MoU berlaku selama periode mata kuliah dan dapat diubah melalui persetujuan tertulis. Pihak dapat menghentikan aktivitas yang tidak aman, tidak etis, atau tidak sesuai scope. Detail legal harus ditinjau oleh pihak universitas sebelum digunakan sebagai dokumen formal.

Status MoU: Sampai laporan ini diperbarui, MoU belum dibuat, belum dibahas untuk persetujuan final, belum disetujui oleh para pihak, dan belum disahkan. Isi di atas hanya merupakan draft konseptual untuk dibahas kemudian.

4.2 4.2 COCOMO: Estimasi Awal

Estimasi berikut menggunakan Basic COCOMO organic sebagai estimasi akademik awal. Ukuran asumsi adalah 2.5 KLOC equivalent untuk vertical slice Python, effort multiplier organic $a = 2.4$, $b = 1.05$, schedule $c = 2.5$, dan $d = 0.38$.

$$E = 2.4(2.5)^{1.05} \approx 6.3 \text{ person-months}$$
$$T = 2.5(E)^{0.38} \approx 4.9 \text{ months}$$

Angka ini bukan klaim durasi aktual tiga mahasiswa. Kode masih kecil, banyak aktivitas bersifat pembelajaran/dokumentasi, dan COCOMO tidak menangkap seluruh reuse standard library maupun kontribusi non-code. Dengan tiga anggota, angka effort kasar tersebut setara rata-rata sekitar 2.1 person-month per anggota sebelum memperhitungkan kalender kuliah.

4.3 4.3 Function Point Analysis

FPA berikut menggunakan pendekatan simplified IFPUG dengan bobot low complexity: External Input (EI)=3, External Output (EO)=4, External Inquiry (EQ)=3, Internal Logical File (ILF)=7, External Interface File (EIF)=5.

Function	Count	Weight	Points
Scenario/controller configuration (EI)	4	3	12
CLI run and experiment execution (EI)	2	3	6
Metrics and comparison output (EO)	5	4	20
State/metrics inspection (EQ)	3	3	9
Simulation state and experiment records (ILF)	3	7	21
Future approved observation input (EIF)	1	5	5
Unadjusted Function Points			**73**

Dengan asumsi general system characteristics total 25 dari skala 0–5, value adjustment factor adalah $0.65 + 0.01(25) = 0.90$. Maka adjusted FP diperkirakan $73 \times 0.90 = 65.7$, dibulatkan menjadi 66 FP. Nilai ini harus direvisi ketika dashboard, database, atau adapter lapangan benar-benar diimplementasikan.

4.4 4.4 Analisis Kebutuhan dan Arsitektur

4.5 Kebutuhan Fungsional

1. Sistem dapat menjalankan simulasi dengan seed dan durasi tertentu.
2. Sistem dapat memilih fixed-time atau queue-aware controller.
3. Sistem dapat menghasilkan demand profile balanced, directional-heavy, dan stress.
4. Sistem memvalidasi setiap proposal sebelum simulated actuation.
5. Sistem menghasilkan arrivals, departures, queue, delay, stops, throughput, waiting, fairness, dan intervention metrics.
6. Sistem dapat menghasilkan raw CSV dan summary JSON untuk matched experiments.

4.6 Kebutuhan Nonfungsional

Reproducibility, local execution, no external dependency, safety-boundary separation, readable code, testability, dan traceability menjadi kebutuhan utama. Performance produksi dan real-time field reliability belum menjadi acceptance claim.

4.7 Arsitektur

Aliran utama adalah demand/observation menuju SimulationState, controller proposal, deterministic validator, simulated actuator, lalu metrics. Future field observation adapter hanya dapat ditambahkan melalui authority-approved gate dan tidak mengubah simulator menjadi pengendali publik secara otomatis.

4.8 4.5 Implementasi Sistem

4.9 Lingkungan Pengembangan

Implementasi menggunakan Python 3 dan standard library. Seluruh kode, test, scripts, dokumentasi, dan report source tersedia pada GitHub repository kelompok: https://github.com/sammtan/MATCS. Repository menggunakan struktur ‘src/mtacs‘ untuk package, ‘tests‘ untuk unittest, ‘scripts/run_experiments.py‘ untuk batch experiment, dan ‘docs‘ untuk design records.

4.10 Domain Models

‘Approach‘, ‘SignalPhase‘, ‘TimingPlan‘, ‘ScenarioConfig‘, ‘SimulationState‘, ‘ControllerProposal‘, ‘ValidationResult‘, dan ‘RunMetrics‘ mendefinisikan kontrak domain. DemandProfile menyediakan probabilitas reproducible untuk balanced, directional-heavy, dan stress scenarios.

4.11 Controllers and Safety

FixedTimeController mengikuti urutan fase terkonfigurasi. QueueAwareController menggunakan queue pressure, waiting age, minimum green, maximum green, dan starvation threshold. Kedua controller hanya mengusulkan fase saat ini atau fase berikutnya. SafetyValidator memeriksa simulation time dan transisi fase.

4.12 Metrics and Batch Runner

MetricsCollector menghitung queue maximum, mean queue, accumulated delay, throughput, fairness index, per-approach waiting, per-approach maximum queue, dan validation interventions. Batch runner menjalankan kombinasi seed, durasi, profile, dan controller lalu menghasilkan ‘raw_runs.csv‘ dan ‘summary.json‘.

4.13 4.6 Pengujian dan Validasi

4.14 Automated Tests

Sebanyak 13 test berhasil dijalankan. Cakupannya meliputi transition, stale proposal, skipped phase, reproducibility, profile/duration matrix, matched controller inputs, stress scale, zero demand, fairness bounds, starvation protection, metrics keys, dan artifact schema.

4.15 Acceptance Result

Hasil final adalah ‘Ran 13 tests ... OK‘. Seluruh test berjalan lokal tanpa dependency tambahan. Zero validation intervention pada batch runs menunjukkan proposal yang dihasilkan controller memenuhi kontrak transisi pada simulator; hasil ini bukan sertifikasi keselamatan dunia nyata.

4.16 4.7 Hasil Eksperimen dan Pembahasan

4.17 Eksperimen

Batch evidence terdiri dari 4 demand profile, 2 durasi (120 dan 600 detik), 5 seed, dan 2 controller, sehingga menghasilkan 80 runs. Perbandingan menggunakan profile, seed, duration, timing, service rate, dan topology yang sama.

Metric	Balanced	North/South Heavy	East/West Heavy
Delay change	+1.84%	-1.33%	-5.77%
Mean queue change	+1.83%	-1.33%	-5.77%
Throughput change/min	-1.40	0.00	+1.60

Fairness fixed → heuristic	0.9782 → 0.9845	0.9747 → 0.9765	0.9153 → 0.9699

Catatan: Profil stress memiliki delay +2.00%, mean queue +2.00%, throughput -0.80 per menit, dan fairness 0.9931 → 0.9960; ditulis terpisah agar tabel tetap terbaca.

4.18 Interpretasi

Queue-aware paling menjanjikan ketika demand tidak seimbang, khususnya east/west-heavy. Namun balanced dan stress short-horizon mengalami regresi kecil. Karena itu, tidak tepat menyimpulkan bahwa heuristik selalu lebih baik. Hasil 600 detik terutama menunjukkan repeatability dan perilaku saturation, bukan bukti kesiapan deployment.

4.19 Kesiapan Fase Berikutnya

MATCS siap memasuki controlled heuristic refinement, statistical evaluation, dan review UML/report. MATCS belum siap untuk klaim AI, calibrated digital twin, field observation, shadow mode, atau real-world actuation. Tahap berikutnya harus menambah seed, confidence interval, statistical test, fairness per approach, dan demand calibration sebelum policy learning dipertimbangkan.

5 BAB 5. KESIMPULAN DAN SARAN (TEMPORARY)

5.1 Kesimpulan Sementara (Temporary)

MATCS telah berkembang dari proposal simulation-first menjadi vertical slice yang dapat dijalankan dan diuji. Sistem memiliki fixed-time baseline, queue-aware heuristic, deterministic safety validator, simulated actuator, richer metrics, batch experiment artifacts, dan dokumentasi UML. Evidence saat ini mendukung manfaat terbatas pada demand directional-heavy, tetapi belum mendukung klaim universal.

5.2 Saran Pengembangan Sementara (Temporary)

Prioritas berikutnya adalah menambah seed dan statistical analysis, memperbaiki pressure scoring dan starvation policy, melakukan peer review kode, menambahkan dashboard setelah kontrak stabil, dan memperbarui COCOMO/FPA berdasarkan scope aktual. Authority package sengaja belum dikembangkan pada fase ini.

5.3 Keterbatasan Saat Ini (Temporary)

Demand masih synthetic, simulator hanya satu persimpangan, travel time dan emissions proxy belum tersedia, intervensi validator belum terpicu pada valid batch, dan hasil tidak dapat diekstrapolasi langsung ke sistem lalu lintas nyata.

Daftar Pustaka

References

[1] B. W. Boehm, *Software Engineering Economics*, Prentice-Hall, 1981.

[2] International Function Point Users Group, *Counting Practices Manual*, release reference.

[3] Object Management Group, *Unified Modeling Language Specification*, OMG.

[4] Group 14, *MATCS System Architecture, Simulation, Controller, and UML Documentation*, 2026.

[5] Python Software Foundation, *Python Standard Library Documentation*.

[6] E. J. Braude and M. E. Bernstein, *Software Engineering: Modern Approaches*, 2nd ed., Waveland Press, 2016.